

TABLE OF CONTENTS

	Page
Table of Contents	i
Chapter 1 Project Report	
1.1 Project Overview	Error! Bookmark not defined.
1.2 System Design	3
1.2.1 System Architecture	4
1.2.2 Data Storage Design	4
1.2.3 Core Components	5
1.3 Development Process	6
1.4 Testing and Evaluation	7
1.4.1 Unit Testing	8
1.4.2 Integration Testing	8
1.4.3 Functional Testing	8
1.4.4 User Acceptance Testing	9
1.5 Project Results	9
1.6 Data Storage Testing	10
1.6.1 Central Storage Testing	10
1.6.2 Local Storage Testing	11
1.7 System Performance Evaluation	11
1.8 Usability Evaluation	11
1.9 Analysis of Testing Results	12
1.10 Limitations and Future Improvement	12

1.1 Project Overview

The **DagonChat** project is a comprehensive, synchronized **Client-Server chat application** engineered to facilitate a structured and reliable flow of information among its participants. Architecturally, the system is built upon a synchronized framework involving three primary operational nodes: Client Side A, the Server Side, and Client Side B. The overarching objective of the application is to provide a modern, intuitive, and distraction-free communication platform that supports both broad, public group discussions and targeted, private one-to-one messaging. By adhering to a minimalist design philosophy, DagonChat ensures that the underlying complexities of network synchronization are abstracted, providing users with a seamless and straightforward experience.

The operational lifecycle of the DagonChat ecosystem is initiated through a rigorous connection protocol. It utilizes a **three-way handshake** to establish a stable network foundation, followed by a mandatory **User Authentication** phase to validate credentials before any communication is permitted. For data persistence, the system leverages **SQLite 3**, opting for a file-based architecture (`dagonchat.db`) to ensure portability and ease of deployment. This "ready-to-run" design is specifically tailored for classroom environments, eliminating the need for complex database server installations on individual workstations. Security remains a cornerstone of the project, with user passwords protected via **salted PBKDF2-SHA256 hashing** before they are committed to the database.

Communication within the application follows the academic **Send-Receive-Reply model**, which ensures that every transaction is accounted for and that the state remains synchronized across all connected endpoints. A critical performance feature of DagonChat is its implementation of **background multi-threading** on the client side; this ensures the user interface remains fluid and never "freezes" while awaiting server responses, even during periods of high network latency. This model effectively manages concurrency by processing logic one transaction at a time per connection, preventing "dangling" requests.

Visually, the application is governed by four core design principles: **clarity, consistency, accessibility, and feedback**. The interface utilizes a professional, soft blue-based color palette (`#3B82F6`) and the **Segoe UI** font to maximize readability. The layout is split into two primary regions: a **left sidebar** for profile and contact management, and a **right content panel** for the active chat feed and input composer. To maintain conversational clarity, DagonChat employs a specialized **Notification System** that diverts presence updates (such as users joining or leaving) to a dedicated notification bell, ensuring the main chat stream remains focused on actual messages.

The technology stack utilizes **C# 12 and .NET 8**, providing a robust and high-performance runtime for real-time messaging. The solution is modularized into four distinct projects—Shared, Server, Client, and Solution Items—to maintain a clean and scalable codebase. Deployment is highly flexible, supporting a "Developer Mode" for

debugging in Visual Studio and a **"Double-Click Mode"** for classroom demos, which uses automated batch files like Run-App.bat to launch the ecosystem instantly. Ultimately, DagonChat represents a sophisticated balance between network engineering and user-centric design.

1.2 System Design

The system design of DagonChat centers on a synchronized **Client-Server architecture** involving active clients and a vigilant server engine. Reliability is ensured through **TCP/IP** (port 9050), while **UDP** (port 9051) facilitates local network discovery. The interaction follows an academic **Send-Receive-Reply model**, utilizing background multi-threading to maintain client-side responsiveness. For **data storage**, the system employs **SQLite 3**, storing identity and messaging history in a portable dagonchat.db file. The solution is organized into three **core components**: a Shared library for protocols, a Server engine for routing and authentication, and a WinForms-based Client. This modular design separates network logic from presentation, ensuring the platform is both scalable and maintainable within a classroom environment.

1.2.1 System Architecture

The **DagonChat** application is built upon a **synchronized Client-Server architecture**, a design specifically chosen to ensure a structured and reliable flow of information among participants. This architecture is operationally divided into three primary components: **Client Side A**, the **Server Side**, and **Client Side B**. The system's connectivity is rooted in the **TCP/IP protocol** (port 9050) for robust, ordered message delivery, while a **UDP discovery responder** (port 9051) allows clients to automatically locate the server within a local network.

At the core of its operational logic is the academic **Send-Receive-Reply cycle**. This structured interaction model ensures that every network transaction is accounted for and synchronized between endpoints. The cycle begins when a client sends a request packet; during this time, the server, which remains in a **receive-blocked idle state**, activates to process the request logic. Once the server issues a reply—whether an acknowledgment (ACK), error message, or data payload—it returns to its idle state, effectively managing concurrency one transaction at a time per connection to prevent **"dangling" requests**.

To maintain high performance and **client-side responsiveness**, DagonChat utilizes **background multi-threading**. By offloading the network receive loop and packet processing to a secondary thread, the application ensures that the **UI thread never "freezes"** while awaiting a server response. This provides a fluid user experience even during periods of high network latency. Furthermore, the server maintains an **active session dictionary** to track online participants and coordinates message routing—broadcasting public messages to all connected clients or forwarding private messages specifically to a designated recipient. This architectural setup provides a scalable foundation that balances network integrity with a modern, responsive user interface.

1.2.2 Data Storage Design

The core persistence layer of **DagonChat** is built upon **SQLite 3**, a robust, serverless, and self-contained relational database engine. By opting for a file-based architecture rather than a traditional client-server database, the system ensures effortless portability and rapid deployment. This design is specifically tailored for classroom environments, as it eliminates the need for complex administrative overhead or external server installations on individual workstations. The database is managed using the `Microsoft.Data.Sqlite 8.0` library, which allows for high-performance reads and writes essential for real-time messaging while keeping the application's footprint minimal.

Upon the first execution of the server, the database is **automatically initialized** in the application's root directory. If the `dagonchat.db` file is missing, the application generates the schema instantly and performs a **seeding operation**, populating the system with default demo accounts to ensure immediate usability. The storage design is centered around two primary relational tables: **Users** and **Chat_Message**.

The **Users table** manages identity and security by storing unique user identifiers, case-insensitive usernames, and **salted PBKDF2-SHA256 password hashes**. It also tracks display names and user roles to ensure proper permission assignment. The **Chat_Message table** archives every interaction, including a unique message ID, sender/recipient usernames, the message content, and a **UTC timestamp** formatted to ISO 8601.

A critical component of this design is the **MessageKind enumeration**, which dictates how the server routes and stores incoming packets. Messages are categorized as **Public (0)** for group chats, **Private (1)** for targeted communication, **System (3)** for internal status, or **Presence (4)** for join/leave notifications. This centralized schema allows the system to distinguish between conversation content and metadata, such as presence updates, which are diverted to a notification bell rather than cluttering the stored chat history.

1.2.3 Core Components

The **DagonChat** solution is architected with a modular project structure designed to ensure the codebase remains maintainable, scalable, and organized through a clear separation of concerns. The system is divided into four distinct components, each serving a specialized role in the communication ecosystem: **DagonChat.Shared**, **DagonChat.Server**, **DagonChat.Client**, and **Solution Items**.

DagonChat.Shared acts as the foundational library for the entire application. It defines the custom communication protocol layer and the network packet structures that both the client and server nodes must use identically to exchange data. By housing shared models, constants, and JSON serialization handlers, this component ensures that information is encoded and decoded consistently across the network, preventing synchronization errors.

DagonChat.Server is the standalone core engine of the system. It is responsible for the critical logic of network management, including handling persistent TCP connections and maintaining the active user state through a specialized UserStore. Beyond basic network routing—where it determines whether to broadcast public messages or forward private ones to specific recipients—the server component also manages mandatory user authentication and interacts directly with the SQLite database to persist message history.

DagonChat.Client represents the graphical presentation layer developed using Windows Forms (WinForms). This component contains all user interface elements and client-side network logic. It functions by mapping UI events, such as clicking a "Send" button, directly to background service request protocols through the ChatClientService. This design ensures the UI thread remains responsive while the background threads handle the complexities of the Send–Receive–Reply cycle.

Finally, **Solution Items** include the scripts and batch files necessary for automated deployment. This component houses PowerShell scripts for environment setup and batch files like Run-App.bat, which facilitate "Double-Click Mode" for rapid execution in classroom settings. Together, these four components form a cohesive environment where the protocol, logic, interface, and deployment scripts work in harmony.

1.3 Development Process

The development of the **DagonChat** application was a structured process focused on balancing high-level technical performance with end-user accessibility. The project utilizes a modern technology stack, with **C# 12** as the primary programming language. This version of C# was specifically selected for its advanced asynchronous programming features, such as **async/await**, which are critical for maintaining a responsive system during high-latency network operations. The application is built on the **.NET 8** framework, providing a robust, long-term support runtime and extensive class libraries that facilitate rapid development and broad compatibility. For the user interface, **Windows Forms (WinForms)** was chosen for its seamless integration with the Windows operating system and its ability to provide access to native controls, ensuring a familiar experience for students and instructors.

The development process emphasized a modular architecture to keep the codebase maintainable and scalable. The solution is partitioned into four distinct projects: **DagonChat.Shared**, **DagonChat.Server**, **DagonChat.Client**, and **Solution Items**. The **Shared library** defines the custom communication protocol layer, network packet structures, and JSON serialization handlers used identically by both nodes to ensure data consistency. The **Server project** houses the core engine responsible for TCP connection handling, maintaining the active UserStore, and managing user authentication. The **Client project** maps UI events directly to background service request protocols, ensuring a clean separation between the presentation and networking layers. Additionally, the solution includes **PowerShell scripts** for automated developer operations, minimizing manual intervention during local deployment.

A central pillar of the development process was the application's design philosophy, which is rooted in **minimalism and functionality**. Developers adhered to four core principles: clarity, consistency, accessibility, and feedback. This led to the selection of **Segoe UI** at specific weights (e.g., 30pt bold for headers) and a professional blue-based color palette (#3B82F6) to create an intuitive, distraction-free atmosphere. Feedback mechanisms, such as hover states and status messages like "Creating account...", were implemented to provide immediate visual cues to the user regarding the results of their actions.

For the data layer, the choice of **SQLite 3** was a strategic development decision intended to simplify deployment in classroom environments. By using the `Microsoft.Data.Sqlite 8.0` library, the development team ensured high performance for real-time messaging without the need for complex external database server installations. Networking logic was developed using **TCP/IP** for reliable, ordered message delivery and **UDP** for low-overhead server discovery on local networks. The implementation of the academic **Send-Receive-Reply model** further ensured transaction integrity and prevented "dangling" requests by processing one transaction at a time per connection.

Finally, the development lifecycle supported multiple execution modes. **Developer Mode** allowed for direct execution from **Visual Studio 2022**, utilizing comprehensive debugging tools and terminal commands like `dotnet run` to test multi-user functionality. For final deployment, a "**Double-Click Mode**" was created, providing batch files such as `Run-App.bat` and `Start-Server.bat`. This allowed users to launch the entire ecosystem with a single click, bypassing the need for deep technical knowledge of the underlying command-line interfaces.

1.4 Testing and Evaluation

Testing for DagonChat is categorized into four primary domains to ensure system integrity. **Unit Testing** focuses on core logic within the `Shared` library, specifically **PBKDF2** hashing and **JSON** serialization. **Integration Testing** evaluates the connection handshake and server address resolution via manual entry or **UDP** auto-discovery. **Functional Testing** verifies registration and login flows against established business rules, such as username uniqueness and password length. Finally, **User Acceptance Testing (UAT)** assesses the interface against core design principles, ensuring that immediate visual feedback and clear navigation provide a positive user experience.

1.4.1 Unit Testing

Unit testing for DagonChat focuses on the isolated verification of critical logic components housed within the `DagonChat.Shared` library. A primary area of focus is the **PBKDF2-SHA256 hashing pipeline**, which ensures that user passwords are correctly salted and hashed before being committed to the **SQLite database**. Additionally, unit tests validate the **JSON serialization and deserialization handlers**,

confirming that complex data objects like `ChatMessage` are accurately converted into length-prefixed byte streams for network transmission. These tests further verify the **MessageKind enumeration** logic, ensuring the system correctly distinguishes between public, private, and system messages. Furthermore, developers test the protocol constants and utility classes to prevent regression errors during development. By rigorously testing these standalone modules, the development team ensures that the core data processing and security protocols remain stable and reliable throughout the application's lifecycle.

1.4.2 Integration Testing

Integration testing for DagonChat focuses on the seamless interaction between the **Client, Server, and Shared projects**. This phase specifically validates the **connection and authentication handshake**, ensuring the client can successfully resolve the server's address through manual entry, saved settings, or **UDP auto-discovery** on port 9051. Tests verify the establishment of a persistent **TCP connection** on port 9050 and the correct execution of the **three-way handshake**. Furthermore, it confirms that the **Send-Receive-Reply cycle** maintains state synchronization across the network, ensuring the server correctly authenticates credentials against the **SQLite database** and registers active sessions.

1.4.3 Functional Testing

Functional testing for DagonChat verifies that the application's core features operate according to defined business rules and user requirements. This includes testing the **registration flow**, ensuring the system enforces the minimum six-character password length and correctly rejects duplicate usernames. The **authentication process** is validated by confirming that valid credentials grant access to the main chat window, while invalid login attempts trigger appropriate error messages to prevent unauthorized access. Additionally, testing covers **messaging functionality**, ensuring public messages broadcast to all users and private messages reach specific recipients. Finally, the **graceful disconnection** process is verified.

1.4.4 User Acceptance Testing

User Acceptance Testing (UAT) evaluates DagonChat against its core design principles: **clarity, consistency, accessibility, and feedback**. Testers confirm that the minimalist interface provides an intuitive experience where all interactive elements are clearly labeled. UAT verifies the **consistency** of the soft blue-based color palette and the **accessibility** of the Segoe UI typography. Crucially, it assesses if **immediate feedback**—such as hover animations and status messages—effectively informs users of their actions, ensuring a distraction-free and professional communication environment.

1.5 Project Results

The successful completion of the **DagonChat** project has resulted in a fully functional, synchronized **Client-Server communication ecosystem** that meets all architectural and design specifications outlined in the project requirements. The primary result is a robust messaging platform capable of facilitating both **public group discussions and private one-to-one interactions** with high reliability and minimal latency.

One of the standout technical results is the reliable connectivity established through the **TCP/IP protocol on port 9050**, which is effectively complemented by a **UDP discovery responder on port 9051**. This dual-protocol approach allows the system to achieve "zero-configuration" setup on local networks, where clients can automatically locate the server without manual IP entry. Furthermore, the implementation of the **academic Send-Receive-Reply model** has proven successful in ensuring transaction integrity; the server effectively manages concurrency by processing logic one transaction at a time per connection, preventing "dangling" requests and maintaining state synchronization across all nodes.

The user interface results demonstrate a high level of usability and aesthetic consistency. By utilizing a **soft blue-based color palette (#3B82F6)** and **Segoe UI typography**, the application achieves a modern, professional look that remains distraction-free. The **split-view layout**—separating contact management in the left sidebar from the active conversation in the right panel—optimizes screen space and user workflow. Additionally, the **notification bell system** successfully diverts presence updates (joins and leaves) away from the main chat feed, ensuring that the primary communication thread remains uncluttered and focused on actual conversation.

From a performance and security perspective, the project delivered a **portable SQLite 3 persistence layer** that initializes and seeds automatically upon first execution. This result ensures that the system is "ready-to-run" in classroom environments without external database dependencies. Security testing confirmed that user credentials are encrypted using a **salted PBKDF2-SHA256 hashing pipeline** before being committed to the `dagonchat.db` file, providing a solid foundation for data protection.

Finally, the project achieved excellent deployment flexibility. The creation of a **"Double-Click Mode"** via automated batch files like `Run-App.bat` allows users to launch the entire ecosystem instantly. These combined results—from the high-performance **multi-threaded client responsiveness** to the intuitive visual bubble design—confirm that DagonChat is a stable, scalable, and user-centric solution for local network communication.

1.6 Data Storage Testing

Data storage testing for DagonChat is essential to verify the integrity, persistence, and security of the system's underlying **SQLite 3 database**. The primary objective of these tests is to confirm that the `dagonchat.db` file reliably serves as a "ready-to-run" record-keeping mechanism for all user identities and communication history.

1.6.1 Central Storage Testing targets the server-side data layer. Tests verify that when the server process initiates, it correctly locates or constructs the necessary database schema and performs a **seeding operation** to populate the system with demo accounts. Critical security testing confirms that user passwords undergo **salted PBKDF2-SHA256 hashing** before storage, ensuring that sensitive credentials are never kept in plain text. Furthermore, the system is tested to ensure that every message processed by the server is successfully archived in the Chat_Message table with a unique ID and a standardized **ISO 8601 UTC timestamp**, facilitating accurate message delivery and audit trails.

1.6.2 Local Storage Testing focuses on the client's ability to persist session configuration. Testing confirms that the client can successfully resolve server addresses using **previously saved settings**, allowing for a faster login experience without re-entering IP addresses. Additionally, these tests verify that the client correctly handles the LoginResultPayload to display the user's role and display name accurately upon entering the main chat interface. This comprehensive testing ensures the data layer remains performant, supporting the rapid reads and writes required for real-time messaging.

1.7 System Performance Evaluation

The performance of **DagonChat** is primarily evaluated based on its ability to maintain **client-side responsiveness** and ensure the user interface never "freezes" during network operations. This is achieved through **background multi-threading**, which offloads the network receive loop and packet processing to secondary threads, allowing the UI thread to remain fluid even during periods of high latency. Additionally, the system leverages the **Microsoft.Data.Sqlite 8.0 library**, which facilitates the **rapid reads and writes** essential for real-time messaging while keeping the server's footprint minimal. Transactional integrity is further optimized by the **Send-Receive-Reply model**, which prevents "dangling" requests by processing logic one transaction at a time per connection. Finally, the use of **.NET 8 and C# 12** provides a high-performance runtime for efficient asynchronous communication.

1.8 Usability Evaluation

DagonChat's usability is anchored in a philosophy of **minimalism and functionality**, emphasizing clarity, consistency, accessibility, and feedback. The interface features a modern aesthetic using the **Segoe UI font** and a professional **soft blue-based color palette** (#3B82F6). Its **split-view layout** optimizes workflow by separating profile and contact management in the left sidebar from the active conversation in the right content panel. To enhance conversational flow, **message bubbles** are color-coded and aligned—right for outgoing and left for incoming. A key usability feature is the **notification bell**, which diverts presence updates away from the main chat feed to prevent clutter. Interactive elements like **multi-line input** and immediate visual cues, such as hover animations, ensure an intuitive and distraction-free user experience.

1.9 Analysis of Testing Results

The **analysis of testing results** for DagonChat confirms that the system effectively balances technical reliability with a user-centric experience. **Unit and integration testing** verified the robustness of the **PBKDF2-SHA256 hashing pipeline** and the **TCP/UDP multi-protocol architecture**, ensuring secure authentication and seamless server discovery. **Functional testing** demonstrated that the system strictly enforces **registration business rules**, such as minimum password length and username uniqueness, while accurately routing both public and private messages.

The performance analysis highlights the success of the academic **Send–Receive–Reply model**, which maintains consistent state synchronization and prevents "dangling" requests. **Background multi-threading** effectively eliminated UI "freezing," providing a fluid experience even during high-latency periods. Furthermore, **usability evaluation** confirmed that diverting presence notifications to a **dedicated bell icon** successfully maintained the clarity of the primary chat feed. These combined results indicate a stable, secure, and intuitive platform ready for immediate deployment in classroom environments.

1.10 Limitations and Future Improvement

While **DagonChat** provides a robust framework for local communication, several technical and functional **limitations** offer clear paths for future enhancement. A significant technical constraint is the **disconnection process**, which currently operates on a "best-effort" basis. If a user's application terminates abruptly or network conditions are poor, the `PacketType.Logout` signal may never reach the server, potentially leaving "ghost" sessions in the active dictionary until a timeout occurs.

From a functional perspective, the current interface includes **placeholders** such as the **"Users" button**, which is intended for future features like user profile editing and extended contact management. Additionally, while the system's centralized enumeration includes an **"Announcement" (Value 2)** kind for teacher-led broadcasts, this feature is reserved and not yet fully implemented in the current version. Future development could also expand the **registration policy** to include diverse administrative roles, moving beyond the single "Client" role to better support the specific needs of a classroom hierarchy. Addressing these limitations will transition DagonChat into a more comprehensive collaborative platform.